

Project III: IOT Application with Raspberry Pi

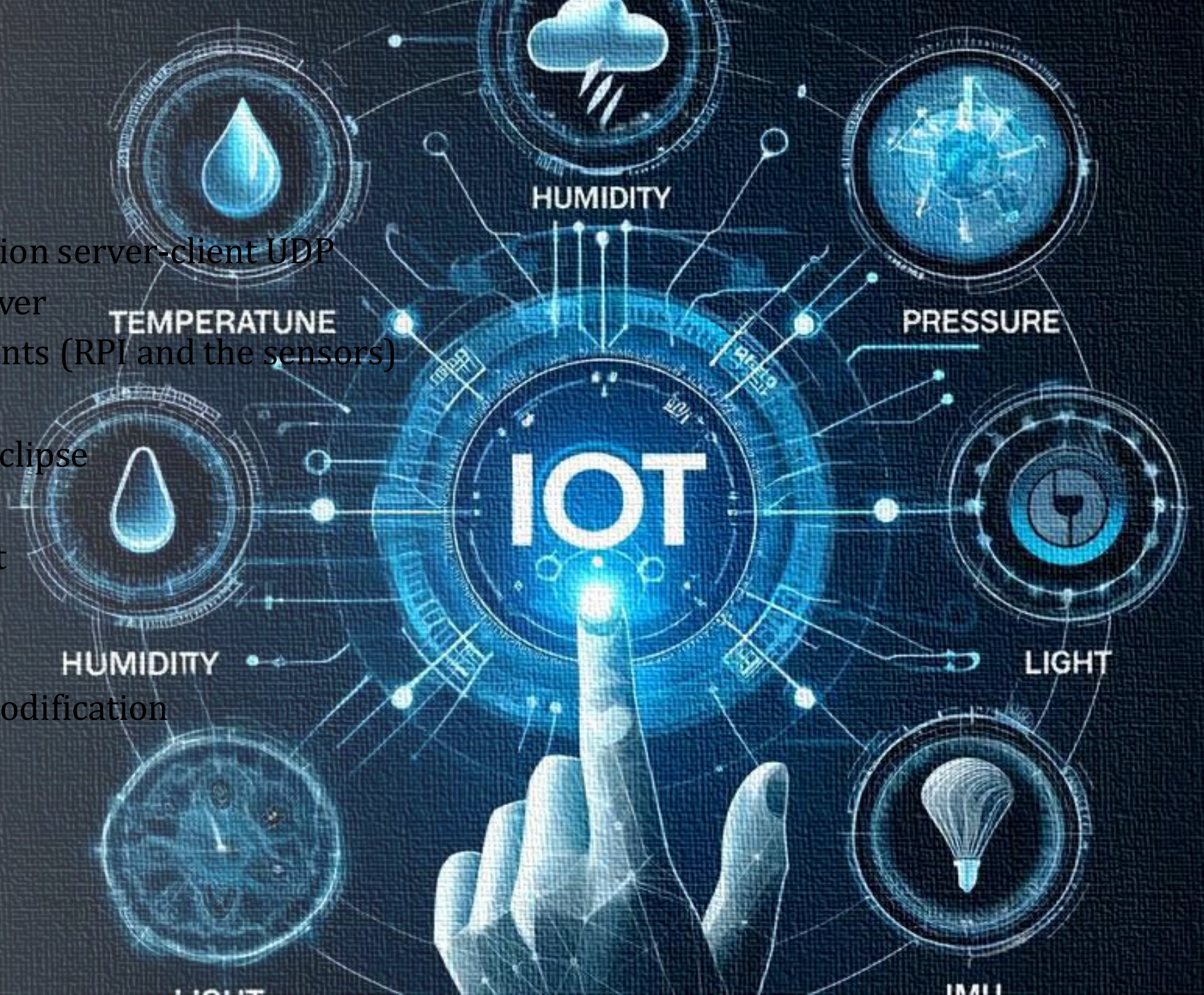
BLANCA LOMA MECHA

YUJIE ZHANG



Content

1. Basic of communication server-client UDP
 - 1.1 Code for the server
 - 1.2 Code for the clients (RPI and the sensors)
2. Configuration with Eclipse
3. Executing the project
4. ThingsBoard
 - 4.1 Installation and modification
 - 4.2 Results
5. Conclusions



1. Basic of communication server-client UDP

1.1 Code for the server

```
int sockfd, portno, n;  
socklen_t clien;  
char buffer[BUFFER_SIZE];  
struct sockaddr_in serv_addr, cli_addr;
```

```
portno = atoi(argv[1]);
```



```
serv_addr.sin_family = AF_INET;  
serv_addr.sin_port = htons(portno);  
serv_addr.sin_addr.s_addr = INADDR_ANY;
```

```
if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)  
{ error("ERROR on binding"); }
```

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);  
if (sockfd < 0) {  
    error("ERROR opening socket"); }  
bzero((char *) &serv_addr, sizeof(serv_addr));
```

```
clilen = sizeof(cli_addr);  
bzero(buffer, BUFFER_SIZE);  
n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1,  
    0, (struct sockaddr *) &cli_addr, &clilen);  
if (n < 0) {  
    error("ERROR receiving from socket"); }  
buffer[n] = '\0';  
printf("Message received from %s:%d\n",  
    inet_ntoa(cli_addr.sin_addr),  
    ntohs(cli_addr.sin_port));  
printf("Raw message:\n%s\n", buffer);
```

1. Basic of communication server-client UDP

1.1 Code for the server

```
if (strcmp(buffer, "Hello Server") == 0) {  
    const char *reply = "Hello RPI";  
    sendto(sockfd, reply, strlen(reply), 0,  
           (struct sockaddr *)&cli_addr, clien);  
    printf("Reply sent: %s\n\n", reply);  
    continue; }  
}
```

```
int sample_count = parse_sensor_message(parse_buffer,  
                                       samples, MAX_SAMPLES);
```

```
printf("\nStatistics:\n");  
calculate_stats("AX", ax, sample_count);  
calculate_stats("AY", ay, sample_count);  
calculate_stats("AZ", az, sample_count);  
calculate_stats("R ", r, sample_count);  
calculate_stats("G ", g, sample_count);  
calculate_stats("B ", b, sample_count);
```

```
printf("Index | AX | AY | AZ | R | G | B\n");  
printf("%5d | %.3f | %.3f | %.3f | %.3f | %.3f | %.3f\n",  
       i, ax[i], ay[i], az[i], r[i], g[i], b[i]);
```

```
printf("%s -> Mean: %.3f | MAX: %.3f | Min: %.3f |  
Desviation: %.3f\n", name, mean, max, min,  
result_variance);
```

```
char reply[128];  
snprintf(reply, sizeof(reply), "ACK: %d samples  
received", sample_count);  
n = sendto(sockfd, reply, strlen(reply), 0,  
           (struct sockaddr *)&cli_addr, clien);  
if (n < 0) {  
    error("ERROR sending to socket"); }  
printf("Reply sent: %s\n\n", reply);
```

1. Basic of communication server-client UDP

1.2 Code for the clients

```
int sockfd, portno, n;  
int i2c_fd;  
socklen_t serverlen;  
char send_buffer[SERVER_BUFFER_SIZE]; char  
reply_buffer[REPLY_BUFFER_SIZE]; struct  
sockaddr_in serv_addr;  
struct hostent *server;  
SensorSample samples[MAX_SAMPLES];
```

```
portno = atoi(argv[2]);  
server = gethostbyname(argv[1]);
```



```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

```
bcopy((char *)server->h_addr,  
      (char *)&serv_addr.sin_addr.s_addr,  
      server->h_length);  
serverlen = sizeof(serv_addr);
```

```
char hello_msg[] = "Hello Server";  
char reply_buffer[256];  
n = sendto(sockfd, hello_msg, strlen(hello_msg), 0,  
           (struct sockaddr *)&serv_addr, serverlen);  
if (n < 0) {  
    error("ERROR sending hello message"); }  
printf("Message sent to server: %s\n", hello_msg);  
bzero(reply_buffer, 256);  
n = recvfrom(sockfd, reply_buffer, 255, 0,  
             (struct sockaddr *)&serv_addr, &serverlen);  
if (n < 0) {  
    error("ERROR receiving server reply"); }  
reply_buffer[n] = '\0';  
printf("Server reply: %s\n\n", reply_buffer);  
if (strcmp(reply_buffer, "Hello RPI") != 0) {  
    printf("ERROR: Server verification failed\n");  
    close(sockfd);  
    exit(1); }  
printf("Connection with server verified  
       successfully\n\n");
```


1. Basic of communication server-client UDP

```
i2c_fd = sensors_init();
```

```
if (read_sensors(i2c_fd, &samples[i]) < 0)
```

```
#define MAX_SAMPLES 10
```

```
build_sensor_message(send_buffer,  
    SERVER_BUFFER_SIZE, samples,  
    MAX_SAMPLES);  
n = sendto(sockfd, send_buffer, strlen(send_buffer),  
0, (struct sockaddr *)&serv_addr, serverlen);
```

```
n = recvfrom(sockfd, reply_buffer,  
    REPLY_BUFFER_SIZE - 1, 0, (struct sockaddr*)  
&serv_addr, &serverlen);
```

1.2 Code for the clients

- what are the server address IP and port?
- Is the server address dynamic or static? Can it be changed?
- What information must be given to the client application about the server configuration?
- What libraries are needed for the Client and Server applications?

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/ioctl.h>
```

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

```
#include <linux/i2c-dev.h>
```

```
#include <linux/i2c.h>
```

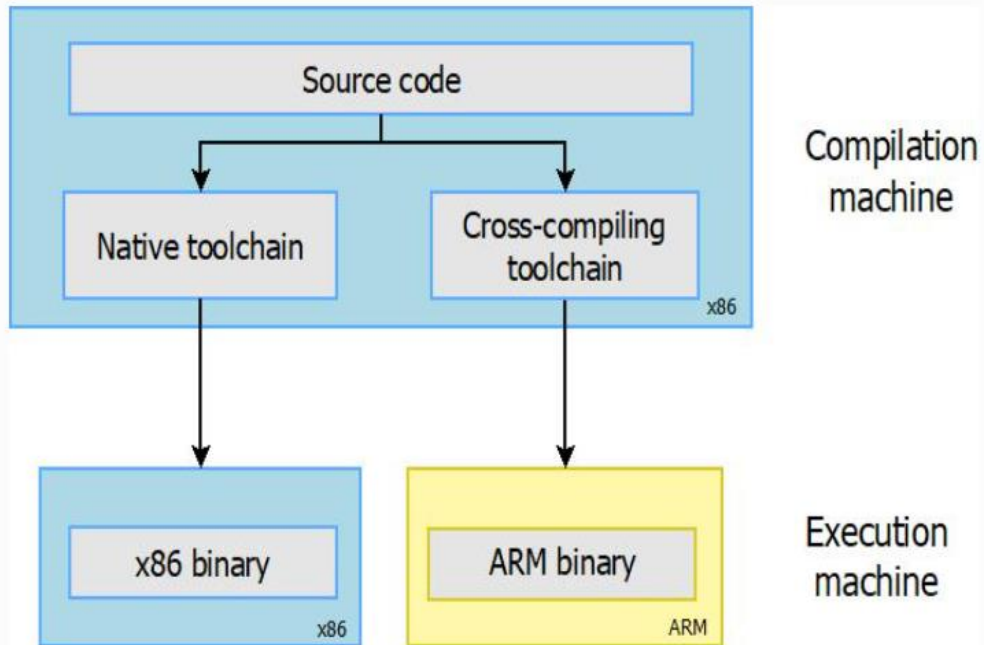
```
#include <netdb.h>
```

2. Configuration with Eclipse

cd Documentos/rpi/build/host
source environment-setup
/home/alumno/eclipse/embedcpp-2026-03/eclipse/eclipse &

Design of Embedded Systems with
RaspberryPI v1.0

PDF



Activate gdbserver

() Additional gcc options
[*] Enable C++ support

[*] gdb
[*] gdbserver

Debug Configuration

Main Arguments Debugger Source Common

Project: client

C/C++ Application: /home/alumno/Escritorio/-Project_Ill_blooma_yzhang_ESD/client/IOTClient_bin

Build (if required) before launching

Build Configuration: Use Active

☐ Enable auto build ☐ Disable auto build

☒ Use workspace settings [Configure Work](#)

Connection: rpi

Remote Absolute File Path for C/C++ Application: /root/IOTClient_bin

server

- IOTServer_bin
- IOTServer.c

client

- IOTClient_bin
- IOTClient.c
- Makefile

GDB debugger: /home/alumno/Documentos/rpi/build/host/bin/aarch64-buildroot-linux-gnu-gdb

GDB command file: .gdbinit

3. Executing the project

```
cd ~/Escritorio/-Project_III_bloma_yzhang_ESD/server
gcc -Wall -Wextra -g -O0 IOTServer.c -lm -o IOTServer_bin
./IOTServer_bin 5000
```

```
/IOTClient_bin 192.168.1.168 5000
```

```
#!/bin/sh

case "$1" in
  start)
    echo "Starting IOTClient..."

    # Esperar a que la red esté lista
    sleep 10

    # Ejecutar cliente automáticamente
    /root/IOTClient_bin 192.168.1.168 5000 &
    ;;

  stop)
    echo "Stopping IOTClient..."
    killall IOTClient_bin
    ;;

  restart)
    $0 stop
    $0 start
    ;;

  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
    ;;
esac

exit 0
```

```
# ./IOTClient_bin 192.168.1.168 5000
TCS34725 Sensor ID: 0x44
Sample 0 -> AX: -0.364 AY: 0.217 AZ: -0.987 R: 84.679 G: 84.817 B: 66.671
```

Automatization file => S99iotclient

```
Sample 4 -> AX: -0.359 AY: 0.220 AZ: -0.984 R: 84.633 G: 84.908 B: 66.606
Sample 5 -> AX: -0.357 AY: 0.215 AZ: -0.990 R: 84.725 G: 84.862 B: 66.570
Sample 6 -> AX: -0.365 AY: 0.220 AZ: -0.998 R: 84.770 G: 84.908 B: 66.576
Sample 7 -> AX: -0.361 AY: 0.219 AZ: -0.997 R: 83.580 G: 85.077 B: 67.808
Sample 8 -> AX: -0.358 AY: 0.217 AZ: -0.981 R: 83.599 G: 85.076 B: 67.811
Sample 9 -> AX: -0.358 AY: 0.224 AZ: -0.984 R: 83.597 G: 85.076 B: 67.901
```

```
Data sent to server:
0,-0.364,0.217,-0.987,84.679,84.817,66.671
1,-0.359,0.217,-0.987,84.634,84.771,66.635
2,-0.362,0.216,-0.990,84.725,84.862,66.707
3,-0.356,0.220,-0.987,84.633,84.908,66.606
4,-0.359,0.220,-0.984,84.633,84.908,66.606
5,-0.357,0.215,-0.990,84.725,84.862,66.570
6,-0.365,0.220,-0.998,84.770,84.908,66.576
7,-0.361,0.219,-0.997,83.580,85.077,67.808
8,-0.358,0.217,-0.981,83.599,85.076,67.811
9,-0.358,0.224,-0.984,83.597,85.076,67.901
Server reply: ACK: 10 samples received
```

```
server$ ./IOTServer_bin 5000
```

```
671
635
707
606
606
570
576
808
811
901
```

```
ation: 0.003
on: 0.003
ation: 0.005
ation: 0.503
ation: 0.106
ation: 0.559
```


4. ThingsBoard

4.1 Installation and modification

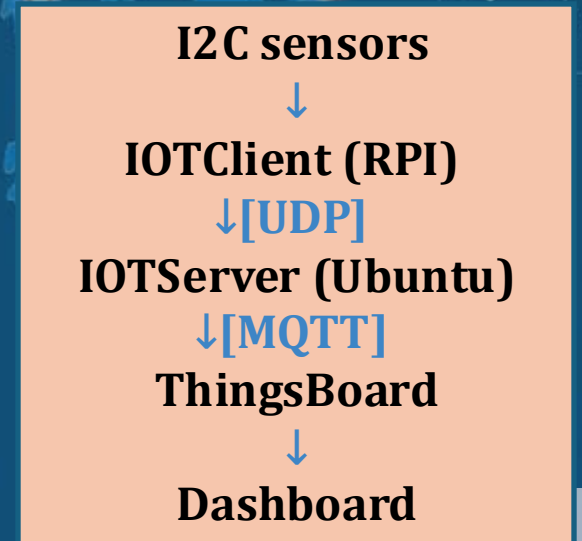
[*] mosquitto

```
# mosquitto_pub -d -h 192.168.1.168 -p 1883 -t v1/devices/me/telemetry -u "TOKEN" -m "{\"ax\":0.12,\"ay\":0.44,\"az\":0.98,\"r\":120,\"g\":80,\"b\":200}"
Client null sending CONNECT
Client null received CONNACK (0)
Client null sending PUBLISH (d0, q0, r0, m1, 'v1/devices/me/telemetry', ... (54 bytes))
Client null sending DISCONNECT
```

```
//MOSQUITTO CONFIGURATION
snprintf(mqtt_cmd, sizeof(mqtt_cmd),
"mosquitto_pub -h 192.168.1.168 -p 1883 -t v1/devices/me/telemetry -u \"TOKEN\" "
"-m \"{\\\"ax\\\":%.3f,\\\"ay\\\":%.3f,\\\"az\\\":%.3f,\\\"r\\\":%.3f,\\\"g\\\":%.3f,\\\"b\\\":%.3f}\"",
samples[i].ax,
samples[i].ay,
samples[i].az,
samples[i].r,
samples[i].g,
samples[i].b);

int ret = system(mqtt_cmd);
if (ret != 0)
{
    fprintf(stderr, "ERROR: mosquitto_pub failed, code=%d\n", ret);
}
```

☐	Last time of update	Key ↑	Value
☐	2026-05-19 17:10:22	ax	0.12
☐	2026-05-19 17:10:22	ay	0.44
☐	2026-05-19 17:10:22	az	0.98
☐	2026-05-19 17:10:22	b	200
☐	2026-05-19 17:10:22	g	80
☐	2026-05-19 17:10:22	r	120



4. ThingsBoard



5. Conclusions

Video testing the codes: [Testing project III video.mov](#)

Thank you

Any question? !!?